GIT | RESUMEN PARA ESTUDIAR

Síntesis del documento «Git» (38 páginas). Conceptos, procedimientos y comandos clave, en español.

1. ¿Qué es Git y para qué sirve?

Git es un sistema de control de versiones distribuido, libre y de código abierto, creado por Linus Torvalds en 2005. Registra qué archivos se modifican, quién hizo los cambios y cuándo, y permite recuperar versiones anteriores. Facilita el trabajo en equipo mediante repositorios locales y remotos.

- Repositorio local: copia del proyecto en el ordenador; su historial se guarda en la carpeta oculta .git.
- Repositorio remoto: copia compartida, por ejemplo en GitHub, GitLab o Bitbucket. Los cambios no se comparten automáticamente: hay que sincronizarlos.
- GitHub: plataforma para alojar repositorios y colaborar mediante ramas y pull requests.

2. Configuración inicial y conexión segura

git config --global user.name "Tu nombre"

git config --global user.email "correo@ejemplo.com"

git config -l · git config --global core.editor "code --wait --new-window"

La configuración puede ser de sistema, usuario (--global) o repositorio; la configuración más específica prevalece. Para conectarse a GitHub se puede usar HTTPS o SSH; el material recomienda SSH.

ssh-keygen -t ed25519 -C "correo@ejemplo.com"

ssh -T git@github.com

Al generar claves SSH, la privada permanece en el equipo y la pública se incorpora a GitHub (Settings → SSH and GPG keys). Se recomienda proteger la cuenta con autenticación de dos factores (2FA).

3. Cómo funciona Git

El flujo local básico tiene tres espacios: directorio de trabajo (archivos editados), staging area o índice (cambios seleccionados para confirmar) y repositorio local (historial de commits). HEAD señala el commit actual, normalmente el último de la rama activa.

Secuencia clave: editar archivos → git add → git commit → git push. Para incorporar cambios remotos se usa git pull.

Estados que distingue el material: untracked (sin seguimiento), sin modificar, modificado, staged (preparado) y committed (confirmado). Cada commit es una instantánea identificada por un hash.

4. Comandos esenciales de trabajo diario

Comando	Utilidad
git clone URL	Clona un repositorio remoto con su historial.
git init	Inicializa un repositorio local y crea .git.
git status	Muestra la rama activa y los cambios pendientes.
git add archivo / git add .	Prepara un archivo o los cambios del directorio actual.
git commit -m "mensaje"	Registra los cambios preparados en un commit local.
git pull origin main	Descarga e integra cambios del remoto en la rama actual.
git push origin main	Publica los commits locales en el remoto.
git log --oneline --graph --decorate --all	Muestra un historial compacto y gráfico de los commits.
git diff / git difftool	Compara cambios por consola o mediante una herramienta visual.
git restore archivo	Descarta cambios no preparados en el archivo.
git restore --staged archivo	Retira el archivo del área de preparación.
git remote -v	Muestra repositorios remotos asociados.

5. Crear un proyecto y publicarlo en GitHub

Si ya existe un repositorio remoto: copiar su URL SSH y ejecutar git clone URL. La copia tendrá el historial y el remoto predeterminado origin.

Si el proyecto nace en local: ejecutar git init, preparar los archivos con git add ., confirmar con git commit -m "mensaje", crear un repositorio remoto vacío y vincularlo mediante git remote add origin URL; finalmente, git push origin main (comprobar el nombre real de la rama).

El archivo .gitignore contiene patrones de archivos que Git no debe rastrear, como *.class o build/.

6. Ramas, HEAD y colaboración

Una rama es un apuntador móvil a un commit. Permite desarrollar una característica o corregir un fallo sin modificar directamente la rama principal. El documento presenta como ejemplo de un entorno DevOps las ramas dev (desarrollo), pre (preproducción) y pro (producción).

Comando	Función
git branch -a	Lista ramas locales y remotas.
git branch nueva	Crea una rama.
git switch nueva / git checkout nueva	Cambia a otra rama.
git switch -c nueva / git checkout -b nueva	Crea la rama y cambia a ella.
git branch -m vieja nueva	Cambia el nombre de una rama.

Comando	Función
git branch -d nueva	Elimina una rama ya integrada.
git push origin nueva	Publica la nueva rama en el remoto.

Pull request: propuesta para fusionar cambios de una rama en otra, con posibilidad de revisar, comentar y discutir los cambios antes de incorporarlos.

7. Merge: fusión y conflictos

git merge rama integra los cambios de «rama» en la rama en la que estás situado; la rama de origen no cambia. Primero hay que cambiar a la rama de destino.

git switch main → git merge funcionalidad

- Fast-forward: si no hay divergencia, Git simplemente avanza el apuntador y el historial queda lineal.
- Merge con --no-ff: obliga a crear un commit de fusión y deja constancia explícita de esa integración.
- Fusión automática: Git combina cambios compatibles sin intervención.
- Conflicto: si dos ramas modifican de forma incompatible la misma zona, hay que editar el archivo, elegir el contenido definitivo y eliminar los marcadores <<<<<<<, ======= y >>>>>>>. Después se ejecutan git add y git commit.

8. Rebase frente a merge

Merge une historias sin reescribir los commits existentes. Rebase traslada los commits de una rama a una nueva base, creando nuevos commits y un historial más lineal. El ejemplo del documento usa git switch myfeature seguido de git rebase main. Como reescribe el historial, conviene tener especial cuidado con commits ya compartidos.

9. Stash: guardar trabajo sin confirmar

git stash guarda temporalmente los cambios para cambiar de tarea y retomarlos después. El documento muestra git stash apply para recuperarlos, git stash list para ver la lista y git stash drop para eliminar una entrada. git stash pop recupera y elimina la entrada; git stash -u también incorpora archivos sin seguimiento.

10. Etiquetas y firma de commits

Etiquetas (tags): señalan puntos importantes del historial, por ejemplo una versión de lanzamiento. git tag las enumera; git tag v1.0 crea una etiqueta ligera; git tag -a v1.4 -m "Versión 1.4" crea una etiqueta anotada con metadatos, que el material recomienda.

Firma: es posible firmar commits mediante GPG, SSH o S/MIME. Para firmar con SSH se configura gpg.format ssh y user.signingkey, se registra en GitHub una clave de tipo «Signing key» y se ejecuta git commit -S -m "mensaje". GitHub puede mostrar la firma como verificada.

11. Repaso rápido: flujo recomendado

1. Obtener los últimos cambios (git pull) → 2. editar → 3. comprobar (git status y git diff) → 4. preparar (git add) → 5. confirmar (git commit) → 6. compartir (git push). Para funcionalidades independientes, trabajar en ramas y fusionar o proponer un pull request tras revisar los cambios.

Idea clave: commit guarda cambios en local; push los envía al remoto; pull trae e integra cambios del remoto.

Fuente: documento facilitado «Git.pdf», apartados «Funcionamento de Git», «Operacións no repositorio local», «Ramas», «Fusión e resolución de conflitos», «Rebase», «Stash», «Etiquetas» y demás secciones.